



Universidade do Minho
Escola de Engenharia

Relatório 1ª Fase

Laboratórios de Informática III

Grupo 13 | 2025/2026

Bruno Magalhães (a110274)

Diogo Azevedo (a109727)

Vera Almeida (a110723)

Índice

1. Introdução -----	3
2. Sistema-----	4
2.1. Estrutura aplicada-----	4
2.2. Funcionamento do projeto-----	5
3. Discussão -----	6
3.1. Modularização -----	6
3.2. Encapsulamento -----	7
3.3. <i>Queries</i> -----	8
3.3.1. <i>Query 1</i> -----	8
3.3.2. <i>Query 2</i> -----	9
3.3.3. <i>Query 3</i> -----	10
3.4. Dificuldades sentidas-----	11
4. Conclusão-----	12
5. Anexos -----	13

1. Introdução

O presente relatório apresentará informações relativas à 1ª Fase do Projeto da Unidade Curricular Laboratórios de Informática III, pertencente ao 2º Ano da Licenciatura em Engenharia Informática, realizada no ano letivo 2025/26, na Universidade do Minho.

O projeto consiste na implementação de uma base de dados em memória, utilizando-se ficheiros .csv fornecidos pela equipa docente, constituídos por dados relativos a viagens (voos, reservas, passageiros, etc.). Através desta base de dados, pretende-se responder a um conjunto de *queries*, explorando conceitos de organização e gestão eficiente da informação.

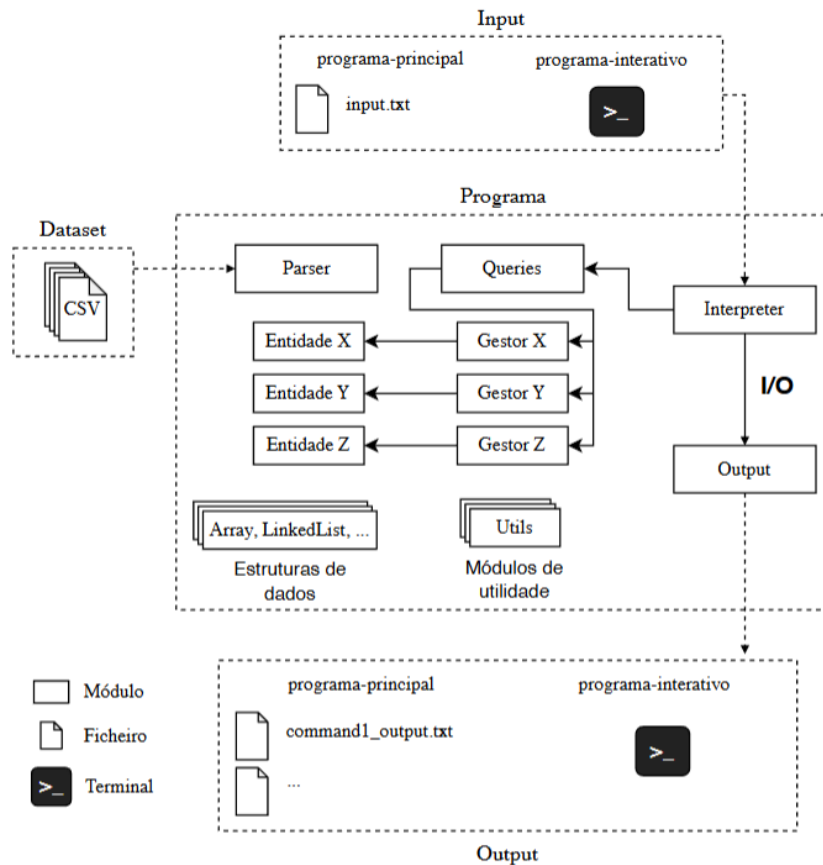
A realização desta primeira fase exigiu uma atenção especial à organização modular e ao encapsulamento do código, assegurando a separação entre os ficheiros (.h) e (.c), de modo a garantir o isolamento adequado entre módulos, assim como a sua segurança.

Nesta 1.ª Fase, desenvolveu-se o *parsing* dos ficheiros de entrada, assim como a validação dos dados dos mesmos, e implementou-se a resolução de três *queries*.

2. Sistema

2.1. Estrutura aplicada

No desenvolvimento do projeto, seguiu-se uma estrutura baseada na arquitetura proposta pelos docentes, organizando o sistema em três componentes principais: uma responsável pela gestão das entidades, outra dedicada à leitura e validação dos *datasets*, e uma terceira encarregue da execução das *queries*.



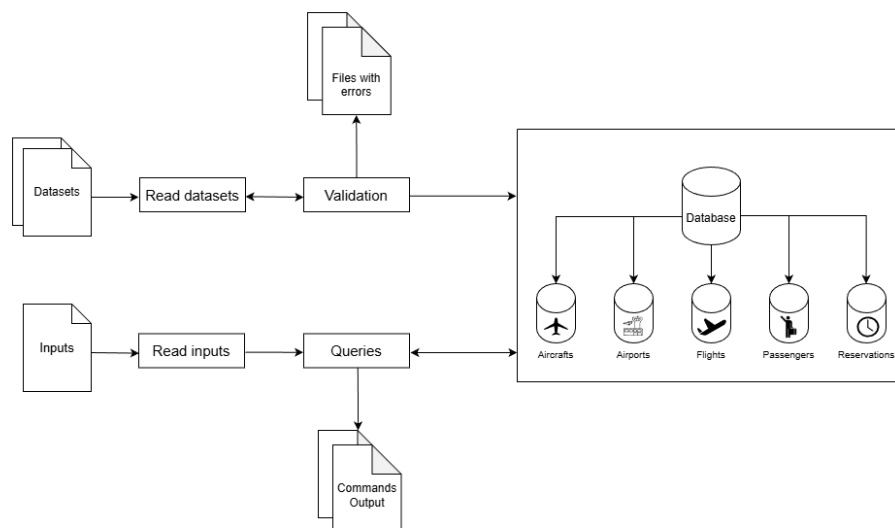
2.2. Funcionamento do projeto

Inicialmente, são criadas as estruturas de armazenamento das entidades – uma *hash-table* para cada uma das entidades (exceto as reservas, que não foram armazenadas, dado que não são utilizadas nesta fase), assim como um *array* dinâmico para os voos, cuja utilização será detalhada posteriormente.

A leitura dos ficheiros é efetuada com base no primeiro argumento do programa, que corresponde ao caminho para os ficheiros .csv dos *datasets*. Para isso, é utilizada uma função genérica de leitura, que se ramifica em cinco funções especializadas – uma para cada entidade. Cada uma destas funções é responsável pela leitura, validação e inserção dos dados nas respetivas estruturas. Se algum registo não cumprir os critérios de validação, a linha é copiada para um ficheiro de erros e não é adicionada à estrutura correspondente.

Concluída a leitura dos dados, é realizado o *parsing* dos comandos a partir do segundo argumento do programa, correspondente ao ficheiro de *inputs*, recorrendo a uma função geral. Esta função identifica o índice da *query* e redireciona a execução para a função específica responsável pela execução dessa *query*. Finalizada a execução de cada uma, é escrito num ficheiro o resultado esperado ou, caso não haja um resultado válido, é escrita uma linha ('\n') para o indicar.

Por fim, após a execução do programa, é efetuada uma limpeza completa da memória, libertando todas as estruturas e dados alocados ao longo do processo, prevenindo *memory leaks* e garantindo a correta terminação do programa.



3. Discussão

3.1. Modularização

A modularização do projeto foi concebida com o objetivo de dividir o sistema em componentes independentes e especializados. Ao separar o código por módulos, reduz-se significativamente a complexidade global do sistema, uma vez que cada parte pode ser compreendida, desenvolvida e testada independentemente.

Durante o desenvolvimento do projeto, foi adotada a seguinte estrutura, organizada em três componentes principais:

- **entities** – Contém a implementação de cada entidade, incluindo as respetivas estruturas de armazenamento, funções de criação, *getters*, *setters* e mecanismos de libertação de memória. O módulo *info_entities.c* agrega e disponibiliza informação auxiliar necessária à articulação entre entidades. Figura 1
- **read_datasets** – Agrupa todos os módulos encarregues da leitura dos datasets associados a cada entidade. Cada ficheiro contém a lógica necessária para interpretar ficheiros .csv, validar os dados (através do módulo *validations.c*) e inserir a informação nas estruturas definidas em *entities*. O ficheiro *read_datasets_main.c* coordena estes processos, garantindo uma leitura dos vários dataset. Figura 2
- **read_inputs** – Agrupa todos os módulos encarregues da interpretação e execução de cada comando. O ficheiro *read_input_main.c* implementa o *parser* dos inputs e direciona cada comando para o módulo correspondente. Cada *query* foi implementada num ficheiro próprio. Figura 3

Para além destes componentes, foram também criadas duas pastas complementares:

- **utils** – Reúne, no ficheiro *utility_functions.c*, funções auxiliares utilizadas em diferentes partes do código, de forma a evitar redundância e centralizar operações comuns a vários módulos.
- **tests** – Pasta dedicada ao programa de testes, utilizada para verificar o correto funcionamento da resolução das *queries* e avaliar o desempenho do programa a nível do uso de memória e do tempo de execução. Contém um ficheiro principal *tests_main.c* e outro com funções auxiliares *tests_functions.c*

Por fim, o ficheiro *main.c* corresponde ao programa principal, responsável por gerir a execução global do sistema.

3.2. Encapsulamento

Neste projeto, também foi adotada uma abordagem centrada no princípio do encapsulamento do código, com o objetivo de assegurar a integridade dos dados e promover uma estrutura de programação mais segura, organizada e de fácil manutenção.

Cada módulo foi desenvolvido com base na separação entre interface pública e implementação interna, através da utilização de ficheiros de cabeçalho (.h) e de implementação (.c) independentes. Os ficheiros de cabeçalho definem estritamente o conjunto de funções necessárias para a interação entre módulos, enquanto os ficheiros de implementação preservam a ocultação dos detalhes internos, incluindo estruturas de dados e lógica operacional. Desta forma, o acesso a variáveis e estados internos, bem como a estruturas de armazenamento dos dados, restringe-se ao código das funções específicas evitando-se assim a manipulação direta de memória.

Adicionalmente, para reforçar a segurança dos dados, foi incorporado o marcador *const* em algumas variáveis. Ao tornar uma determinada variável numa constante, as funções que a recebem como parâmetro, apenas podem lê-lo sem efetuar quaisquer modificações. Esta metodologia reduz substancialmente a probabilidade de erros devido a modificações inadvertidas de valores críticos.

3.3. Queries

Com o objetivo de avaliar o desempenho do código desenvolvido, foi efetuada a análise do tempo médio de execução de cada *query* em diferentes máquinas.

Para tal, os testes foram realizados em dois *Lenovo IdeaPad* (configurações 1 e 2) e num *MacBook Pro* (configuração 3). As configurações possuem as seguintes especificações:

	Config. 1	Config. 2	Config. 3
CPU	AMD Ryzen 7 5700U	Intel Core i7-13620H	M3 Pro
RAM	16GB	16GB	18GB

Os dados apresentados sobre cada *query* correspondem a uma média de 5 execuções consecutivas numa dada configuração. Não foram apresentados os valores do uso de memória, uma vez que são consistentemente iguais (cerca de 135MB) e independentes do *input* e da configuração.

3.3.1. Query 1

Tem como objetivo listar o resumo de um aeroporto, apresentando o código, nome, cidade, país e tipo de aeroporto, de acordo com o identificador recebido como argumento. Figura 4

Dado que cada aeroporto possui um código único, a utilização de uma *hash-table* para armazená-los torna o tempo de execução desta *query* praticamente instantâneo, justificando a nossa escolha de estrutura de armazenamento. Para além disso, a quantidade de aeroportos a armazenar é bastante reduzida (inferior a 10^4), pelo que o impacto da *hash-table* na utilização de memória é insignificante. A seguir encontram-se os tempos de execução da *query*, repetida com 40 inputs diferentes:

<i>Dataset sem erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.003	0.003	0.004
Tempo Total (ms)	0.111	0.101	0.125

<i>Dataset com erros</i>	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.005	0.003	0.005
Tempo Total (ms)	0.197	0.091	0.193

3.3.2. Query 2

Tem como objetivo apresentar o Top N aeronaves com maior número de voos, por ordem decrescente. Em casos de empate no número de voos, a ordenação deve ser feita por ordem alfabética do identificador da aeronave. Esta *query* recebe como argumento o número de aeronaves a incluir no resultado e pode ainda receber um filtro opcional de fabricante. Figura 5

Para facilitar a implementação, foi adicionado um novo campo na estrutura da aeronave, correspondente ao número de voos (*flight_count*). Assim, à medida que o *dataset* de voos é processado, incrementa-se este campo na aeronave correspondente (se o voo é válido e não cancelado). Posteriormente, foi criada uma estrutura auxiliar (*InfoAircraft*) que contém apenas o identificador e o número de voos da aeronave, de forma a simplificar a manipulação e ordenação destes dados, já que eram os únicos relevantes para esta *query*. Estas estruturas foram armazenadas num *array* ordenado de tamanho N.

Com estas medidas tomadas, uma única iteração sobre a *hash-table* das aeronaves resolve o problema apresentado pela *query*. Durante esta iteração, são adicionadas ao *array* de N estruturas as aeronaves. Quando havia o filtro de fabricante, apenas aquelas que continham esse fabricante eram adicionadas. Quando o *array* fica completamente preenchido, aplica-se o algoritmo de *QuickSort* para ordenar as aeronaves pelo número de voos e, por fim, sempre que era encontrada uma nova aeronave com mais voos do que a que tinha o valor mínimo no *array* (ou, caso tivessem o mesmo número de voos, se a nova aeronave era lexicograficamente inferior), esta era inserida, substituindo esse mínimo.

A utilização do parâmetro de *flight_count* nas aeronaves tem um impacto irrelevante na utilização de memória, dado que existem apenas cerca de 10^3 aeronaves, mas permite uma eficiente execução da *query* 2, visto que evita a necessidade de percorrer a (enorme, visto que existem aproximadamente 10^6 voos) estrutura de armazenamento dos voos. Tendo isto em conta, o tempo de execução desta *query* encontra-se a seguir:

<i>Dataset</i> sem erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.041	0.084	0.033
Tempo Total (ms)	1.628	3.345	1.332

<i>Dataset</i> com erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	0.077	0.081	0.034
Tempo Total (ms)	3.075	3.246	1.350

3.3.3. Query 3

Tem como objetivo identificar o aeroporto com mais partidas entre duas datas fornecidas como argumento (data inicial e final). O resultado consiste no aeroporto com mais partidas e o número total de partidas correspondentes. Tal como na *query 2*, empates são resolvidos por ordem alfabética do identificador (neste caso, o *flight_id*). Figura 6

Aquando da leitura do *dataset* dos voos, além da *hash-table*, criou-se também um *array* dinâmico ordenado que contem todos os voos ordenados por ordem crescente da respetiva data de partida real (*actual_departure*). Além disso, para auxiliar na execução da *query*, uma nova *struct* (*InfoAirport*) que contém o identificador do aeroporto e o número de partidas (*departure_count*) do mesmo foi utilizada.

A execução da *query* inicia-se com a procura de um voo no *array* cuja data se encontre dentro do intervalo especificado. A partir desse ponto, o programa percorre o *array* em ambas as direções (esquerda e direita) até ultrapassar os limites das datas. Durante este processo, a informação de cada aeroporto encontrado é armazenada numa *hash-table* auxiliar da seguinte forma: se o aeroporto ainda não existe na *hash-table*, é adicionado à mesma com *departure_count* = 1, caso contrário, incrementa-se esse campo. Além disso, durante este processo, é mantida uma variável *max*, que guarda o aeroporto com o maior número de partidas até ao momento de modo a evitar uma necessidade de percorrer a *hash-table* auxiliar ao fim da execução da *query*. Sempre que um aeroporto ultrapassa o valor atual de *max*, a variável é atualizada. Os tempos de execução para esta *query* são os seguintes:

<i>Dataset</i> sem erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	43.577	30.624	24.375
Tempo Total (ms)	1743.06	1224.96	975.01

<i>Dataset</i> com erros	Config. 1	Config. 2	Config. 3
Tempo Médio (ms)	59.303	30.535	24.695
Tempo Total (ms)	2372.1	1221.39	987.81

3.4. Dificuldades sentidas

No que respeita aos desafios enfrentados, a principal dificuldade durante a execução desta primeira fase baseou-se na gestão do tempo de execução e no controlo da utilização de memória. Através da análise das *queries*, verificou-se que o desempenho do sistema era fortemente condicionado pela seleção das estruturas de dados e pela organização da informação, o que exigiu uma procura contínua de métodos mais eficientes de armazenamento e do processamento da informação.

Relativamente à otimização do consumo de memória, as implementações iniciais armazenavam uma quantidade excessiva de informação nas estruturas de dados, resultando num consumo desnecessariamente elevado. Para resolver este problema, procedeu-se à identificação e remoção dos campos não utilizados durante a execução das *queries*, assim como a utilização de tipos reduzem o uso de memória, quando possível (como por exemplo usar *char* ou *short* ao invés de *int*), mantendo apenas os elementos essenciais.

Para além disso, a *query* 3, em particular, uma vez que a implementação de inicial de uma *hash-table* para os voos, com cerca de 10^6 elementos, levando à necessidade de percorrê-la por completo sempre que a *query* 3 era executada, tornou a sua execução extremamente ineficiente (mais de metade do tempo de execução do programa era derivado da execução da *query* 3). Para mitigar esta limitação, optou-se pela criação de um *array* ordenado pela data de *actual_departure*, permitindo a aplicação de pesquisas binárias de complexidade logarítmica e restringindo o processamento ao subconjunto de dados relevante. A conjugação deste *array* com uma *hash-table* auxiliar destinada à contagem de partidas permitiu uma redução significativa da complexidade temporal, passando de uma procura linear para uma procura logarítmica. Consideramos que, embora o uso de memória tenha aumentado significativamente com a introdução do *array* (cerca de 20MB, ou um aumento de 20%), a redução do tempo de execução da *query* 3 compensava este custo.

4. Conclusão

Em suma, com a realização desta primeira fase, conseguimos aprender novos conceitos e técnicas nomeadamente o *parsing* de ficheiros, a validação de dados, bem como, a modularização e o encapsulamento do código.

Apesar das nossas dificuldades, o desenvolvimento do projeto foi concluído dentro dos prazos previstos, sem atrasos, abrangendo todas as tarefas obrigatórias.

No entanto, existem ainda possíveis melhorias que poderemos aplicar, sendo um dos nossos objetivos a ter em conta durante o desenvolvimento da 2ª fase.

5. Anexos

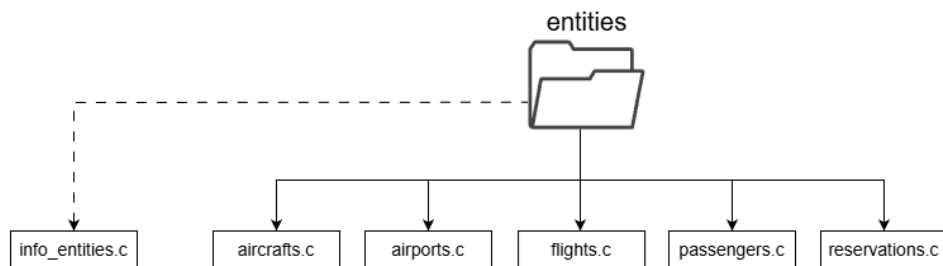


Figura 1: Esquema da organização interna da diretoria *entities*.

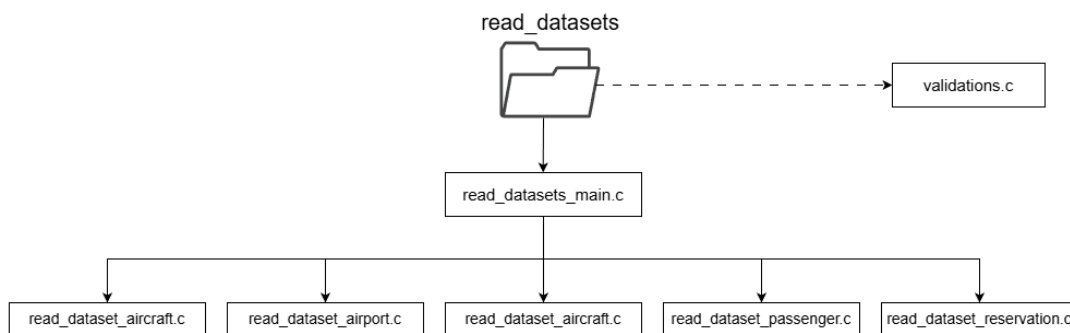


Figura 2: Esquema da organização interna da diretoria *read_datasets*.

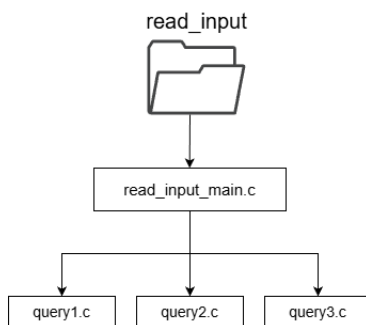


Figura 3: Esquema da organização interna da diretoria *read_input*.

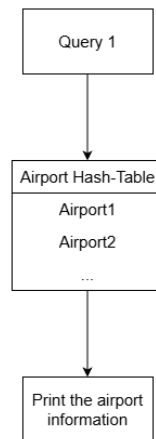


Figura 4: Esquema do funcionamento da *query* 1.

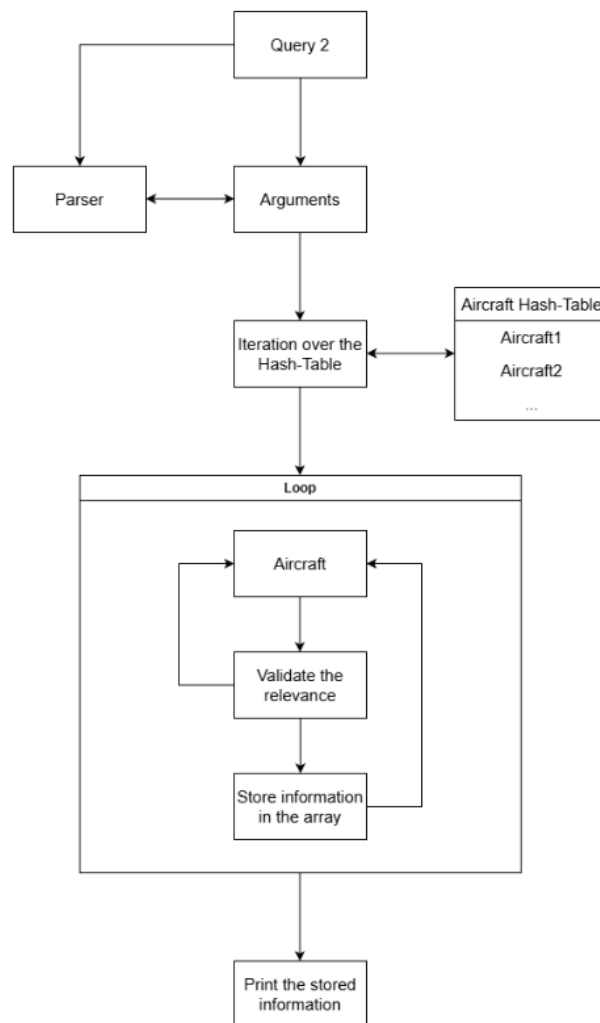


Figura 5: Esquema do funcionamento da *query* 2.

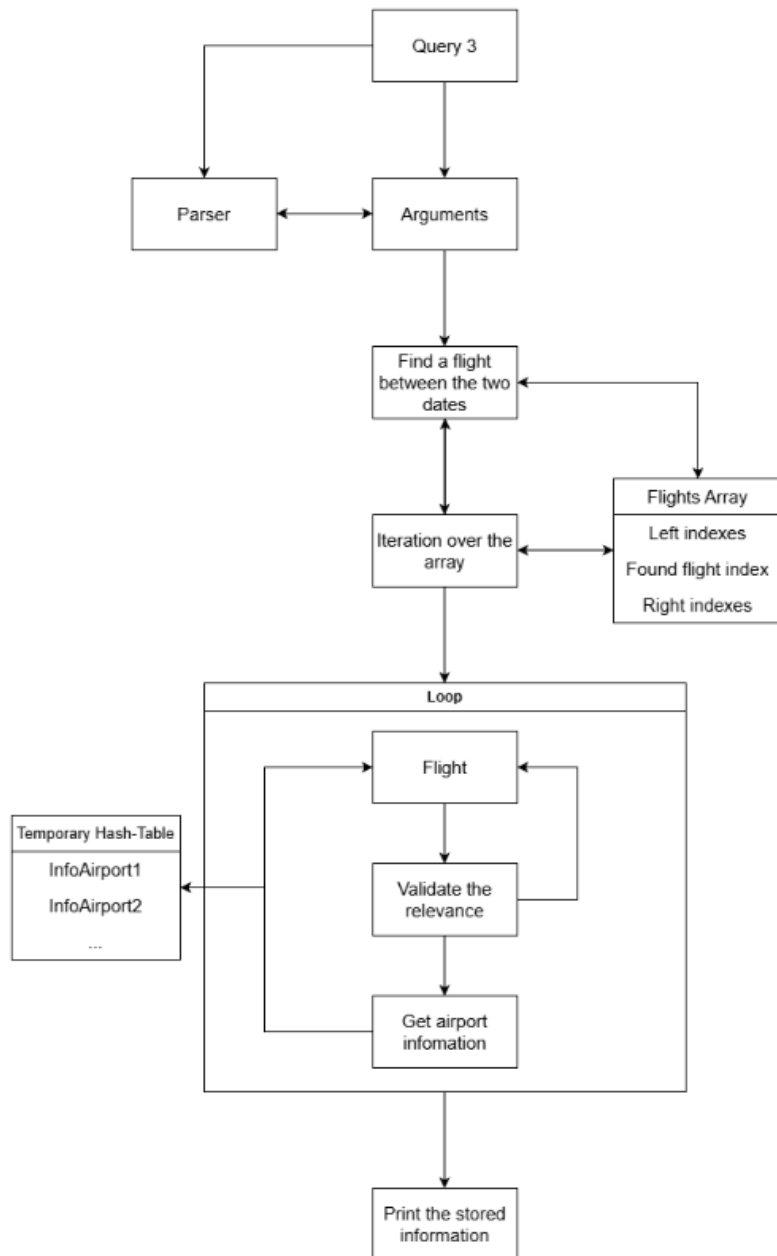


Figura 6: Esquema do funcionamento da query 3.